Voyage-AI — Documentation Suite Index & Navigation Guide

Document Reference: `docs/reports/README.pdf` / `docs/reports/README.md`
System Version: 0.1.0 (Phase 1 Baseline)
Publication Date: October 21, 2026
Author: Senior Software Architect & Technical Documentation Specialist
Status: Implementation-Verified Documentation Index

---

1. Overview & Purpose of the Documentation Suite

This documentation suite provides a comprehensive, implementation-verified technical audit of the **Voyage-AI** repository (`g:\Projects\JFS\Voyage-AI`). The platform is an all-in-one travel discovery and itinerary planning application built as a modern decoupled modular monolith uniting a React 18 / TypeScript single-page application with a Java 17 LTS / Spring Boot 4.1.1 REST backend and a MySQL 8.0 relational database.

Unlike speculative requirement documents, all seven reports in this suite are grounded strictly in the **current verified source code, executed automated test suites, and empirical host runtime evidence** as of October 21, 2026.

---

2. Master Report Directory & Reading Order

The documentation suite is structured into seven specialized reports available in both PDF and Markdown formats within `docs/reports/`:

Report File (PDF / MD)	Title	Target Audience	Primary Focus & Coverage
README.pdf	**Documentation Suite Index**	All Evaluators	High-level map, report abstracts, status definitions, and reading guide.
architecture.pdf	**System Architecture & Technical Design**	Architects & Engineers	Modular monolith structure, verified technology stack, component diagrams, runtime sequence, data flow, ERD, and API contracts.
methodology.pdf	**Engineering Methodology & Workflows**	Team Leads & Developers	Phased engineering lifecycle, end-to-end operational workflows, data pipeline, provider integration rules, and testing layers.
project-report.pdf	**Comprehensive Project Report**	Academic & Project Reviewers	Standalone technical paper: problem statement, scope boundaries, implementation summary, empirical test results, and strategic vision.
implementation-status.pdf	**Verified Implementation Status**	QA & Product Managers	Evidence-based inventory classifying every component as verified, unverified, partially implemented, or planned.
testing-and-quality.pdf	**Testing Strategy & Software Quality**	QA Engineers & Auditors	Multi-tiered test pyramid, verified test execution logs (27/27 passed), database isolation guards, and quality defect analysis.

Report File (PDF / MD)	Title	Target Audience	Primary Focus & Coverage
risks-and-roadmap.pdf	**Technical Risks & Prioritized Roadmap**	Engineering Leads & Product	Risk register (RSK-01 to RSK-08), P0-P3 prioritized backlog, actionable task breakdowns, and phase exit gates.

Recommended Reading Sequences:

- **For Technical Evaluators & System Auditors:** `README.pdf` → `architecture.pdf` → `testing-and-quality.pdf` → `implementation-status.pdf`.
- **For Academic Reviewers & Handover Stakeholders:** `README.pdf` → `project-report.pdf` → `methodology.pdf` → `risks-and-roadmap.pdf`.
- **For Contributing Developers:** `README.pdf` → `architecture.pdf` → `methodology.pdf` → `testing-and-quality.pdf` → `risks-and-roadmap.pdf`.

---

3. Current Project State & Documentation Scope

3.1 Implemented & Verified Baseline (Phase 1)

- **Frontend Presentation Layer (`frontend/`):** Tactile, accessible single-page application built with React 18, TypeScript, Vite, and Tailwind CSS. Implements responsive navigation, category tabs, real-time search filtering, and 5-tab detail inspection modals across benchmark Indian destinations (Jaipur, Varanasi, Munnar, Hampi).
- **Backend Service Layer (`backend/`):** Spring Boot 4.1.1 application running on Java 17 LTS on host port `8081`. Exposes `/api/v1/health`, `/actuator/health`, and `/api/v1/weather` with CORS validation, global exception mapping, and Open-Meteo meteorological integration with data provenance tracking.
- **Relational Persistence Foundation (`infra/`):** MySQL 8.0 containerized orchestration on host port `3307` (avoiding host 3306 conflicts) with Flyway baseline migration (`V0__baseline.sql`).
- **Automated Verification:** 100% of runnable automated test suites verified passing in this session (20/20 backend tests; 7/7 frontend unit tests).

3.2 Status Classification Key

- **Implemented and verified:** Code exists and verified by passing test execution or live inspection.
- **Implemented but unverified:** Code exists, but execution was not performed in this session.
- **Partially implemented:** Code exists for a portion of the feature, but the end-to-end flow is incomplete.
- **Planned:** Documented architectural target without active implementation code.
- **Deprecated or superseded:** Replaced or eliminated from current architecture.

---

4. Canonical Repository Documentation References

This documentation suite references and consolidates the following canonical project artifacts:
- Architecture.md: High-level architectural overview and boundaries.
- README.md: Repository quick start, prerequisites, and developer setup.
- PRD.md: Product Requirements Document.
- FRD.md: Functional Requirements Document.

- TDD.md: Technical Design Document.
- IMPLEMENTATION_PLAN.md: Phased development roadmap.
- JWT_AUTH_DESIGN.md: Stateless JWT authentication specification.
- DATA_AUTHENTICITY_AND_PROVIDER_PLAN.md: Data provenance and external provider compliance rules.
- PRE_IMPLEMENTATION_AUDIT_v1.1.md: Exhaustive environment and requirements baseline audit.